



Domain-Driven Design (DDD) e Arquitetura de Software — TP2

Nome: Guilherme de Morais China Costa

Data: 24/08/2026

Agregados, eventos de domínio e Event Sourcing

1. Explique de forma sucinta qual a razão de criar Agregados.

Criamos agregados para agrupar objetos que precisam obedecer às mesmas regras de negócio. Em vez de deixar qualquer parte do sistema alterar os dados livremente, existe uma raiz que controla essas mudanças. Isso ajuda a manter o modelo organizado e evita estados inválidos.

2. O que significa “consistência transacional”?

Consistência transacional significa que uma transação leva o sistema de um estado válido para outro, respeitando as regras do negócio e as restrições dos dados. No Pet Friends, uma consulta só pode ser confirmada se tiver pet, veterinário e horário válidos. A operação ocorrer por inteiro ou não ocorrer é outra propriedade: a atomicidade.

3. Cite as 4 propriedades cruciais que definem transações.

São as propriedades ACID: **Atomicidade**, que faz a transação ocorrer por inteiro ou ser desfeita; **Consistência**, que preserva as regras do banco e do negócio; **Isolamento**, que evita interferência indevida entre transações simultâneas; e **Durabilidade**, que mantém os dados mesmo depois de uma falha.

4. O que são “invariantes de negócio”?

São regras que precisam ser verdadeiras sempre que o agregado é salvo. No Pet Friends, uma consulta não pode existir sem um pet, um veterinário e um horário. Essas condições não são apenas validações de tela: fazem parte da regra do domínio e devem ser protegidas pelo agregado.

5. Porque um agregado só deve ter acesso a outro agregado pelo ID?

Porque carregar o objeto inteiro de outro agregado aumenta o acoplamento e pode levar a alterações em cadeias muito grandes. Guardar somente o ID deixa clara a fronteira de cada agregado. No Agendamento, basta conhecer o identificador do Pet e do Veterinário; os detalhes deles pertencem aos seus próprios contextos.

6. Crie um trecho de código Java de uma entidade que represente um agregado e faça referência a outro agregado dentro do escopo do projeto Pet Friends.

No exemplo abaixo, `Agendamento` é a raiz do agregado. Ele aponta para `Pet` e `Veterinário` apenas por UUID, sem carregar esses agregados dentro dele.

```
public final class Agendamento {
    private final UUID id;
    private final UUID petId;           // agregado Pet
    private final UUID veterinarioId;  // agregado Veterinário
    private final LocalDateTime horario;
    private Status status;

    public Agendamento(UUID petId, UUID veterinarioId,
                       LocalDateTime horario) {
        this.id = UUID.randomUUID();
        this.petId = Objects.requireNonNull(petId);
        this.veterinarioId = Objects.requireNonNull(veterinarioId);
        this.horario = Objects.requireNonNull(horario);
        this.status = Status.PENDENTE;
    }
}
```

Eventos de domínio

7. Elabore um trecho de código Java que mostre um método de negócio com a previsão de publicação de um evento de domínio dentro do escopo do projeto Pet Friends.

O agendamento começa pendente. O método `confirmar` protege a invariante de que uma consulta só pode ser confirmada uma vez: se o status não for pendente, a operação é interrompida. Quando essa regra é atendida, o status muda para confirmado e o evento `ConsultaAgendada` é registrado. Depois, o caso de uso publica os eventos pendentes.

```
public final class Agendamento {
    private final UUID id;
    private final UUID petId;
    private final UUID veterinarioId;
    private final LocalDateTime horario;
    private final List<DomainEvent> eventos = new ArrayList<>();

    public Agendamento(UUID petId, UUID veterinarioId,
                       LocalDateTime horario) {
        this.id = UUID.randomUUID();
        this.petId = Objects.requireNonNull(petId);
        this.veterinarioId = Objects.requireNonNull(veterinarioId);
        this.horario = Objects.requireNonNull(horario);
        this.status = Status.PENDENTE;
    }

    public void confirmar() {
        if (status != Status.PENDENTE)
            throw new IllegalStateException("Consulta já foi processada");
        status = Status.CONFIRMADO;
        eventos.add(new ConsultaAgendada(id, petId, veterinarioId,
                                         horario, Instant.now()));
    }

    public List<DomainEvent> retirarEventos() {
        List<DomainEvent> pendentes = List.copyOf(eventos);
        eventos.clear();
        return pendentes;
    }
}

public Agendamento executar(UUID petId, UUID veterinarioId,
                             LocalDateTime horario) {
    Agendamento agendamento = new Agendamento(petId, veterinarioId, horario);
    agendamento.confirmar();
    agendamento.retirarEventos().forEach(publicador::publicar);
    return agendamento;
}
```

8. O que é “evento de domínio”?

Evento de domínio é o registro de algo importante que já aconteceu no negócio. O nome deve aparecer no passado, como `ConsultaAgendada` ou `VeterinarioInativado`. Ele comunica um fato para outros componentes sem obrigar o agregado a conhecer quem vai reagir a esse fato.

9. Crie um trecho de código Java que mostre uma abstração de um objeto do tipo “evento de domínio”.

A interface abaixo define informações básicas que qualquer evento da aplicação precisa oferecer: uma identificação única e a data em que ocorreu.

```
public interface DomainEvent {
    UUID eventId();
    Instant occurredOn();
}
```

10. Crie um trecho de código Java que mostre a implementação de um “evento de domínio” dentro do escopo do projeto Pet Friends.

Este evento representa a confirmação de uma consulta entre um pet e um veterinário. O `record` funciona bem porque o evento é um retrato imutável do que aconteceu.

```
public record ConsultaAgendada(
    UUID agendamentoId, UUID petId, UUID veterinarioId,
    LocalDateTime horario, Instant occurredOn, UUID eventId
) implements DomainEvent {
    public ConsultaAgendada(UUID agendamentoId, UUID petId,
        UUID veterinarioId, LocalDateTime horario,
        Instant occurredOn) {
        this(agendamentoId, petId, veterinarioId, horario,
            occurredOn, UUID.randomUUID());
    }
}
```

Código-fonte das questões 6, 7, 9 e 10:

https://github.com/GuiC0506/infnet/tree/master/ddd_e_arquitetura_de_software/TP2/petfriends-eventos

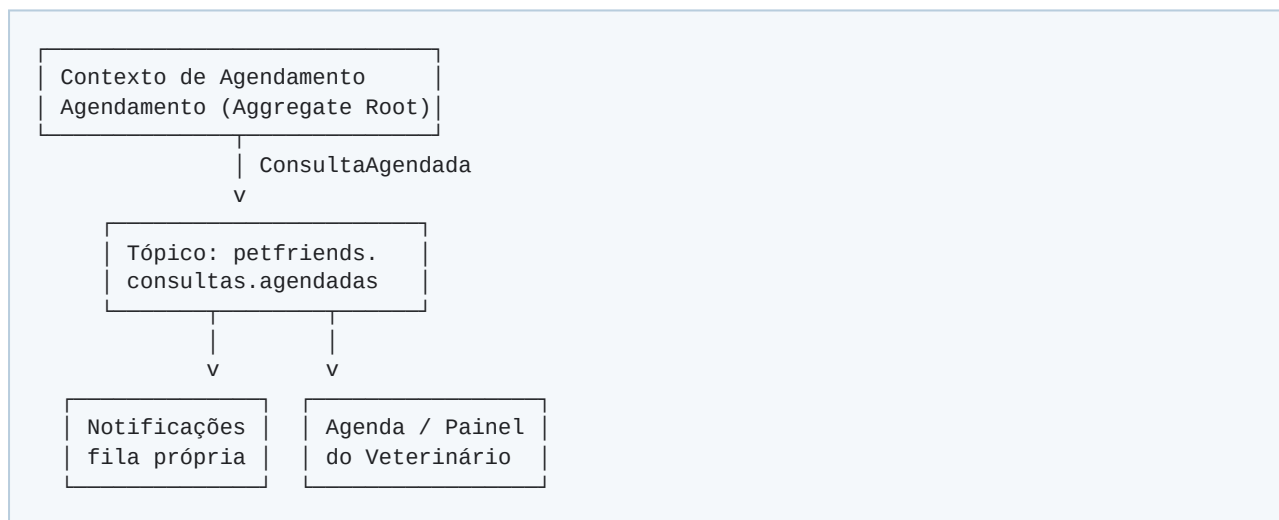
Mensageria e arquitetura de eventos

11. Qual é a diferença entre filas e tópicos e como estes elementos funcionam em conjunto?

Uma fila entrega cada mensagem a apenas um consumidor. Ela é útil quando vários trabalhadores dividem tarefas, como o envio de lembretes. Já um tópico publica a mesma mensagem para vários interessados. Assim, quando uma consulta é agendada, o mesmo evento pode chegar ao serviço de notificações, à agenda do veterinário e à área de indicadores. Em uma arquitetura real, um tópico pode distribuir o evento e cada consumidor pode ter sua própria fila para processá-lo no seu ritmo.

12. Dê um exemplo de solução de arquitetura para publicação de eventos de domínio dentro do escopo do projeto Pet Friends (ideal um desenho).

Uma solução seria publicar `ConsultaAgendada` após a confirmação do agregado. O Agendamento envia o evento para um tópico; cada contexto interessado recebe uma cópia sem depender diretamente do serviço que criou a consulta.



Dessa forma, o serviço de Agendamento não precisa chamar diretamente nenhum dos consumidores. Se um novo contexto quiser reagir ao evento no futuro, basta assinar o tópico.

13. Explique qual a finalidade de uma Event Store no contexto de eventos de domínio.

Uma Event Store é um repositório especializado para armazenar o histórico de eventos do domínio. Seus registros são imutáveis: depois que um evento é gravado, ele não deve ser alterado nem apagado; novos fatos são acrescentados à sequência. Assim, além de saber o estado atual de um agregado, é possível entender como ele chegou até ali, por exemplo, quando uma consulta foi criada, reagendada ou cancelada.

14. O que é Event Sourcing e como ele se diferencia da persistência tradicional em bancos de dados relacionais? Explique como os eventos salvos são usados para recuperar o estado atual de um Agregado.

No Event Sourcing, a fonte principal de dados é a sequência de eventos, e não somente uma linha com o estado atual. Em uma persistência relacional tradicional, o agendamento poderia ser salvo com status "CONFIRMADO"; as mudanças anteriores costumam ser substituídas. Com Event Sourcing, seriam guardados eventos como `ConsultaAgendada`, `ConsultaReagendada` e `ConsultaCancelada`. Para recuperar o agregado, o sistema lê os eventos na ordem em que ocorreram e aplica cada um deles, reconstruindo o estado final. Quando há muitos eventos, pode-se salvar periodicamente um snapshot para acelerar essa reconstrução.